

QUIZ MASTER V1 APPLICATION

STUDENT NAME: Rohit Rai

STUDENT EMAIL ID: 23f1000362@ds.study.iitm.ac.in

DESCRIPTION

The Quiz Master application is a web-based platform built using Flask that serves as an exam preparation tool for students across multiple courses. The app is designed to have two roles: **Admin** (Quiz Master) and **Users** (Students). The admin has full control over the content, while users can attempt quizzes to test their knowledge. The platform allows admins to create subjects, chapters, quizzes, and questions, while users can register, log in, and track their quiz performance.

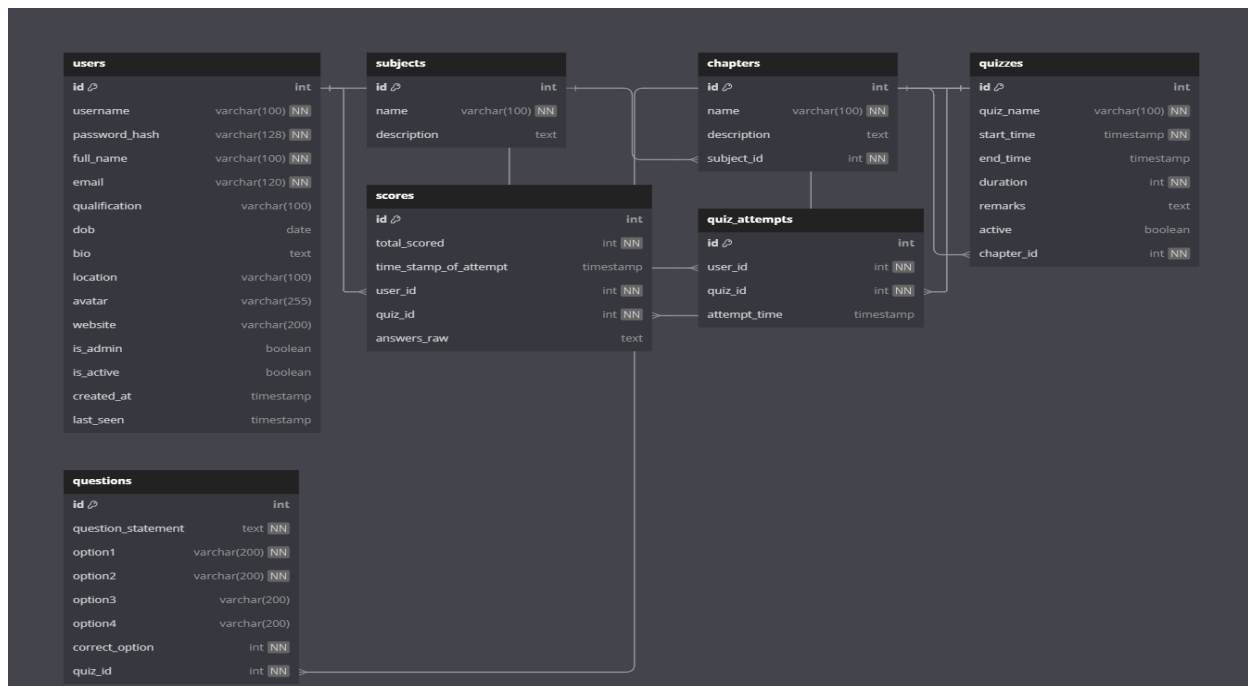
The application is simple yet powerful, offering features like quiz management, user authentication, and performance tracking—all with a clean and responsive interface.

TOOLS USED

- **Flask:** A lightweight framework used for building the backend of the application.
- **Jinja2:** A templating engine to dynamically generate HTML pages.
- **SQLite:** A lightweight database system used for storing data locally.
- **Bootstrap:** Integrated for responsive and visually appealing front-end design.
- **HTML/CSS:** Used for structuring and styling the web pages.
- **Chart.js:** Used to visualize quiz performance data through charts and graphs.

DATABASE SCHEMA DESIGN

The database schema is designed to efficiently manage hierarchical educational content and user interactions. It uses a relational structure with well-defined relationships between entities to ensure data integrity.



USER Table: Stores user information such as ID, username (email), password, full name, qualification, and date of birth.

ADMIN Table: Stores admin-specific information (admin role is predefined).

SUBJECT Table: Contains subject details like name and description.

CHAPTER Table: Stores chapter information linked to subjects.

QUIZ Table: Includes quiz details such as chapter association, date of quiz, time duration, etc.

QUESTION Table: Contains all quiz questions with multiple-choice options and correct answer indicators.

SCORE Table: Tracks user quiz attempts, timestamps, and scores.

SYSTEM FUNCTIONALITY:

Authentication and Role Management

- Admin login: Predefined admin account with superuser privileges (no registration required).
- User registration: Fields include username (email), password, full name, qualification, and date of birth.
- Role-based access control: Admin dashboard for content management; user dashboard for quiz attempts.

Admin Dashboard

- Create/Edit/Delete subjects and chapters under subjects.
- Create/Edit/Delete quizzes under chapters with configurable parameters like duration and scheduling.
- Add/Edit/Delete multiple-choice questions within quizzes (only one correct option).
- View all registered users along with their details.
- Search functionality for users, subjects, quizzes, or questions.

User Dashboard

- Browse available subjects and quizzes based on chapters.
- Attempt quizzes in an MCQ format with a timer (optional).
- Get instant feedback after completing a quiz (correct/incorrect answers).
- View past quiz attempts along with scores.

Performance Tracking

- Quiz scores are stored in the database for future reference.
- Users can view summary charts showing their progress across different subjects and chapters.

Analytics

Admins can view summary statistics of quizzes attempted by users using visualizations created with Chart.js.

IMPLEMENTATION HIGHLIGHTS

Architecture and Features

The project is organized using Flask's **Blueprints**, enabling modular development by dividing functionalities into separate files and folders. The main application logic resides in the application folder, which contains subfolders for controllers, templates, and static assets.

- **Controllers:** Located in the blueprints folder, each file (e.g., admin_bp.py, user_bp.py, auth_bp.py) handles specific routes and functionality for admin, user, authentication, API interactions, and statistics.
- **Templates:** HTML files are stored in the templates folder, categorized into subfolders like admin, user, and auth. A shared layout file (base.html) ensures consistency across pages.
- **Static Files:** The static folder contains CSS (e.g., Bootstrap), JavaScript, and other assets for styling and responsiveness.
- **Database:** SQLite serves as the database backend, with schema defined programmatically in models.py.
- **Utilities:** Common functions and reusable logic are implemented in utils.py, while decorators for route protection are defined in decorators.py.

VIDEO LINK: <https://drive.google.com/file/d/1SRHE3cbzozNPZmW2kCcX0LJFq6zIUDBp/view?usp=sharing>

This project demonstrates the implementation of a complete quiz management platform that fulfils the requirements for an exam preparation tool. With its intuitive interface and robust functionality, Quiz Master provides both admins and users an efficient way to manage educational content and track learning outcomes effectively.